Guía de iptables - Mesa 2

Oct 19, 2026 · @rack

Nos conviene **iptables-nft**, la versión que Ubuntu Server 24.04 ya trae por defecto: usa nftables por debajo, funciona bien en servidores normales (físicos) y no hay que instalar nada raro. Esta guía explica cada paso con su comando, para las 5 máquinas de la red `192.168.50.0/24`.

Cuál iptables nos conviene

En Ubuntu 24.04 existen tres opciones. Elegimos la primera.

Opción	Qué es	¿Nos conviene?
iptables-nft	Los mismos comandos `iptables`, pero guarda las reglas en nftables. Es el predeterminado de Ubuntu 24.04.	**Sí.** Es lo que usa el sistema y funciona bien en servidores normales (físicos).
iptables-legacy	La versión antigua, con su propio sistema de reglas.	No. Si se mezcla con la anterior, las reglas quedan separadas y se confunden.
nftables (`nft`)	El sistema nuevo, con otra forma de escribir las reglas.	No para este proyecto. Hay que aprender otra sintaxis y no ganamos nada.

Regla de oro: usar **solo una** versión en todas las máquinas y no usar `ufw` al mismo tiempo.

Paso 1 y 2: verificar la versión e instalar

1. Ver qué versión de iptables está activa. Debe decir `(nf_tables)`:

```
iptables --version
```

Salida esperada: `iptables v1.8.10 (nf_tables)`

2. Si dijera `(legacy)`, cambiarla a nft:

```
sudo update-alternatives --set iptables /usr/sbin/iptables-nft
sudo update-alternatives --set ip6tables /usr/sbin/ip6tables-nft
```

3. Instalar iptables y el paquete que guarda las reglas al reiniciar. Si pregunta si guardar las reglas actuales, responder que sí:

```
sudo apt update
sudo apt install -y iptables iptables-persistent
```

4. Comprobar que `ufw` no esté activo. Debe decir `inactive`:

```
sudo ufw status
```

Si estuviera activo, se apaga con `sudo ufw disable`.

Paso 3: reglas base (en las 5 máquinas)

Estas reglas bloquean todo lo que entra, menos lo necesario. El orden importa: primero se permite, y la regla `DROP` va al final para no quedarnos sin acceso por SSH.

1. Permitir el tráfico interno de la propia máquina:

```
sudo iptables -A INPUT -i lo -j ACCEPT
```

2. Permitir las respuestas a conexiones que la máquina ya abrió:

```
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

3. Permitir SSH solo desde la red del proyecto:

```
sudo iptables -A INPUT -p tcp --dport 22 -s 192.168.50.0/24 -j ACCEPT
```

4. Permitir ping, para poder probar la red:

```
sudo iptables -A INPUT -p icmp -j ACCEPT
```

5. Por último, bloquear todo lo demás que entre:

```
sudo iptables -P INPUT DROP
```

Paso 4: reglas de cada máquina

Después de las reglas base, cada máquina abre solo los puertos de su servicio. Son los puertos por defecto de cada programa. La base de datos es MariaDB, que usa el puerto 3306.

Máquina	Puerto	Para qué	Comando
Nodo-cerebro y Nodo-backup	4505 y 4506 TCP	Salt (las demás máquinas se conectan aquí)	`sudo iptables -A INPUT -p tcp -m multiport --dports 4505,4506 -s 192.168.50.0/24 -j ACCEPT`
Nodo-cerebro y Nodo-backup	9000 TCP	Página web de Graylog	`sudo iptables -A INPUT -p tcp --dport 9000 -s 192.168.50.0/24 -j ACCEPT`
Nodo-cerebro y Nodo-backup	12201 TCP y 1514 UDP	Graylog recibe los registros (logs) de las demás máquinas	sudo iptables -A INPUT -p tcp --dport 12201 -s 192.168.50.0/24 -j ACCEPT y sudo iptables -A INPUT -p udp --dport 1514 -s 192.168.50.0/24 -j ACCEPT
Nodo-cerebro (donde corre dnsmasq)	67 UDP y 53 UDP/TCP	dnsmasq: DHCP (reparte las IP) y DNS	sudo iptables -A INPUT -p udp --dport 67 -j ACCEPT, sudo iptables -A INPUT -p udp --dport 53 -s 192.168.50.0/24 -j ACCEPT y sudo iptables -A INPUT -p tcp --dport 53 -s 192.168.50.0/24 -j ACCEPT
Nodo-datos	3306 TCP	Base de datos, solo para cerebro y backup	`sudo iptables -A INPUT -p tcp --dport 3306 -s 192.168.50.10 -j ACCEPT` y otra igual con `-s 192.168.50.11`
Nodo-voip	5060 UDP	Asterisk: llamadas entre las extensiones 201 y 202	`sudo iptables -A INPUT -p udp --dport 5060 -s 192.168.50.0/24 -j ACCEPT`
Nodo-voip	10000 a 20000 UDP	Audio de las llamadas	`sudo iptables -A INPUT -p udp --dport 10000:20000 -s 192.168.50.0/24 -j ACCEPT`

Máquina	Puerto	Para qué	Comando
Nodo-respaldos	55414 TCP	Página web de UrBackup	`sudo iptables -A INPUT -p tcp --dport 55414 -s 192.168.50.0/24 -j ACCEPT`
Nodo-respaldos	35621 a 35623 TCP y UDP	UrBackup habla con las demás máquinas	`sudo iptables -A INPUT -p tcp -m multiport --dports 35621,35623 -s 192.168.50.0/24 -j ACCEPT` y `sudo iptables -A INPUT -p udp -m multiport --dports 35622,35623 -s 192.168.50.0/24 -j ACCEPT`

En las máquinas que usan UrBackup como cliente (todas las demás), también hay que abrir los puertos 35621 TCP y 35622 UDP con los mismos comandos.

Filtrado con dnsmasq. El DHCP no tiene filtro de IP de origen porque las máquinas piden dirección sin tener una todavía. El filtro se hace en dnsmasq: en `/etc/dnsmasq.conf` se agrega una línea `dhcp-host=MAC,IP,nombre` por cada nodo y la línea `dhcp-ignore=tag:!known`. Así solo reciben IP las 5 máquinas conocidas y cada una conserva siempre la misma dirección de la tabla.

Paso 5 y 6: guardar, probar y deshacer

1. Guardar las reglas para que no se borren al reiniciar:

```
sudo netfilter-persistent save
```

2. Ver las reglas activas. Los números de cada regla sirven para borrarla si hay error:

```
sudo iptables -L -n -v --line-numbers
```

3. Probar desde otra máquina de la red que el puerto abierto responde (cambiar la IP y el puerto):

```
nc -zv 192.168.50.10 4505
```

4. Reiniciar una máquina y confirmar que las reglas siguen ahí:

```
sudo reboot
sudo iptables -L -n
```

Si algo sale mal, para volver a dejar todo abierto:

```
sudo iptables -P INPUT ACCEPT
sudo iptables -F
```

Para borrar una sola regla, usar su número: `sudo iptables -D INPUT 5`.